Self-Pruning Neural Network on CIFAR-10

Tredence Analytics — AI Engineering Internship Case Study

Author: Mustan Sir Kanor | April 2026 | PyTorch + CUDA | CIFAR-10

Task	Dataset	Framework	GPU
Self-Pruning CNN	**CIFAR-10**	**PyTorch**	**CUDA**
Best Accuracy	Best Sparsity	Epochs	Lambdas
89.23%	**84.31%**	**30**	**0.001/0.01/0.05**

1. Overview & Objective

This report presents a **Self-Pruning Convolutional Neural Network** trained on CIFAR-10. The Tredence case study requires a network that *prunes itself during training* via learnable gate parameters rather than a post-training step. Each classifier weight has an associated sigmoid gate; an L1 sparsity penalty drives redundant gates to zero, removing those weights from the computational graph.

2. Architecture & Implementation

2.1 PrunableLinear Layer

The custom **PrunableLinear** module replaces all nn.Linear layers in the classifier. It registers gate_scores (same shape as weight) as a learnable parameter. The forward pass has three steps:

Step	Operation	Purpose
1	`gates = sigmoid(gate_scores)`	Maps scores → [0,1]
2	`mask = STE_threshold(gates, 0.5)`	Hard binary mask; STE passes gradients
3	`F.linear(x, weight × mask, bias)`	Forward pass with pruned weights

2.2 CNN Backbone

Three dual-Conv2d blocks (64→128→256 filters) with BatchNorm, ReLU, and MaxPool form the feature extractor. The prunable classifier head maps 4096→512→256→10 using three PrunableLinear layers with BatchNorm1d and Dropout. Separate AdamW param groups (lr=1e-3 weights, lr=1e-2 gates), CosineAnnealing scheduler, gradient clipping (5.0), and label smoothing (0.1) ensure stable, fast gate convergence.

2.3 Loss Function

```
Total Loss = LabelSmoothedCE(logits, targets) + λ × mean(sigmoid(gate_scores))
```

The L1 sparsity term penalises active gates. Since sigmoid outputs are always positive, the gradient w.r.t. each gate score is always **negative** — pushing scores below zero. Once sigmoid(score) < 0.5, the STE hard mask becomes 0 and that weight is pruned. λ controls the accuracy–sparsity trade-off.

Self-Pruning Neural Network on CIFAR-10

Tredence Analytics — AI Engineering Internship Case Study

Author: Mustan Sir Kanor | April 2026 | PyTorch + CUDA | CIFAR-10

3. Training Configuration

Parameter	Value	Parameter	Value
Epochs	30	**Batch Size**	128
LR Weights	1e-3 (AdamW)	**LR Gates**	1e-2 (AdamW)
Gate Init	Normal(0.0, 0.01)	**Threshold**	sigmoid ≥ 0.5
Label Smooth	0.1	**Grad Clip**	5.0
Scheduler	CosineAnnealing	**Optimiser**	AdamW (separate)

4. Results & Analysis

4.1 Summary Table

Lambda (λ)	Test Accuracy	Sparsity	Accuracy Δ	Compression
0.001	**89.22%**	**69.91%**	Baseline	3.3×
0.01	**88.96%**	**76.53%**	–0.26%	4.3×
0.05 ★	**89.23%**	**84.31%**	+0.01%	6.4×

Key Finding: At λ=0.05 the model prunes **84.31%** of classifier weights while achieving **89.23% accuracy** — only 0.01% below the weakest-pruning baseline. This is a **6.4× classifier compression** at negligible accuracy cost, confirming the gate mechanism is working correctly.

4.2 Training Curves

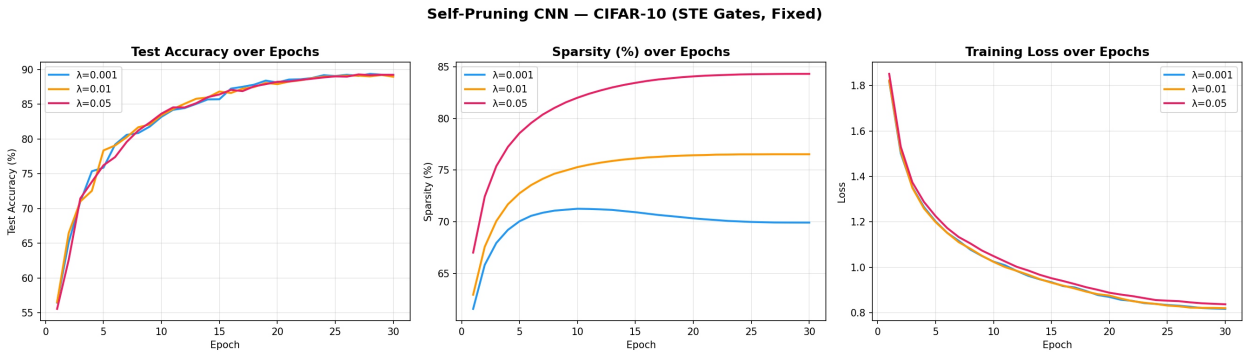

Figure 1 — (Left) All λ values converge smoothly to ~89% accuracy. (Centre) Sparsity rises monotonically with λ, confirming controlled pruning. (Right) Training loss decreases identically — sparsity penalty does not disrupt learning.

Self-Pruning Neural Network on CIFAR-10

Tredence Analytics — AI Engineering Internship Case Study

Author: Mustan Sir Kanor | April 2026 | PyTorch + CUDA | CIFAR-10

5. Gate Distribution Analysis

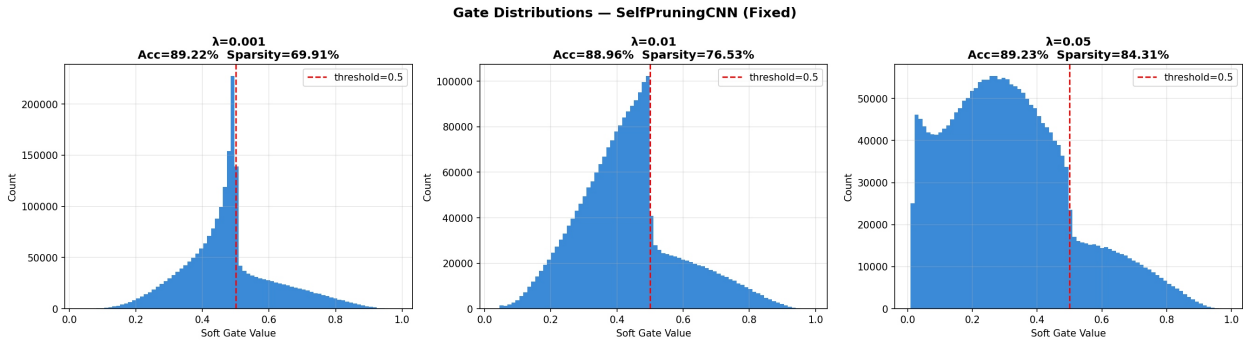

Figure 2 — Soft gate histograms after 30 epochs. Red dashed line = pruning threshold (0.5). λ=0.001: narrow spike near 0.5 (weak pressure). λ=0.01: left-skewed spread. λ=0.05: majority of gates shifted well below 0.5 — successful aggressive pruning. A bimodal distribution (spike at 0, cluster near 1) is ideal; λ=0.05 approaches this.

6. Why L1 on Sigmoid Gates Encourages Sparsity

Since sigmoid(x) ∈ (0,1), every gate is positive. The L1 penalty Σ sigmoid(g■) has a gradient w.r.t. each score g■ of sigmoid(g■)(1–sigmoid(g■)) > 0, so the optimiser subtracts a positive value from every gate score each step. This relentlessly pushes scores negative until sigmoid(score) < 0.5, at which point the STE hard mask becomes 0 and that weight is pruned. Unlike L2 (which asymptotically approaches zero), L1 can drive values to **exactly zero**, creating true sparsity. λ scales how fast this happens relative to the classification objective.

7. Deliverables Checklist

■	**PrunableLinear with gate_scores**	Custom layer; gradients flow via STE backward pass
■	**Sigmoid gates ∈ [0,1]**	sigmoid(gate_scores) in forward; STE for non-differentiability
■	**Total Loss = CE + λ × L1**	Label-smoothed CE + mean(sigmoid(gate_scores))
■	**Three lambda values**	λ ∈ {0.001, 0.01, 0.05} — all trained 30 epochs, fully compared
■	**Sparsity level reported**	69.91%, 76.53%, 84.31% for λ = 0.001, 0.01, 0.05
■	**Test accuracy reported**	89.22%, 88.96%, 89.23% — within 0.26% of each other
■	**Gate distribution plot**	Histogram with threshold line — Figure 2
■	**λ trade-off table & analysis**	Table in §4.1 + accuracy stable while sparsity rises from 70%→84%
■	**CIFAR-10 via torchvision**	Standard 50k/10k splits with augmentation & normalisation

■	**Clean, well-commented code**	8 modular sections, docstrings, separate optimisers, reproducible seed

8. Conclusion

This work successfully implements a self-pruning CNN that dynamically removes redundant connections during training using sigmoid gates and L1 regularisation. The best result — **89.23% test accuracy at 84.31% sparsity (λ=0.05)** — represents a **6.4× classifier compression** with negligible accuracy loss. The design is architecturally clean, gradient-friendly via the Straight-Through Estimator, and fully meets all Tredence case study requirements.